Snowflake w praktyce

Analityka przesyłek kurierskich: FLATTEN, tabele tymczasowe, widoki zmaterializowane i dynamiczne tabele

Warsztat praktyczny · 4 moduły z weryfikacją

Poziom: średniozaawansowany (znajomość SQL, podstawy Stage i COPY INTO)

Spis treści

1. Scenariusz i plan warsztatu ... 3
1.1. Scenariusz biznesowy ... 3
1.2. Przykładowy dokument JSON ... 3
1.3. Architektura rozwiązania ... 3
1.4. Agenda ... 4
1.5. Wymagania ... 4
1.6. Oczekiwane liczby – ściąga dla trenera ... 4
2. Przygotowanie środowiska (start od zera) ... 6
3. Moduł 1. Dane półstrukturalne i FLATTEN ... 10
Weryfikacja modułu 1 ... 12
4. Moduł 2. Tabele tymczasowe i transient – kontrolowane ładowanie ... 14
Trzy rodzaje tabel w Snowflake ... 14
Weryfikacja modułu 2 ... 16
5. Moduł 3. Widoki zmaterializowane ... 17
Weryfikacja modułu 3 ... 19
6. Moduł 4. Dynamiczne tabele ... 20
Weryfikacja modułu 4 ... 22
7. Podsumowanie i sprzątanie ... 24
7.1. Co zbudowaliśmy ... 24
7.2. Kiedy co wybrać ... 24
7.3. Pytania kontrolne ... 24
7.4. Sprzątanie ... 24

1. Scenariusz i plan warsztatu

1.1. Scenariusz biznesowy

Firma kurierska **ParcelGo** obsługuje sklepy internetowe w sześciu dużych miastach w Polsce. System śledzenia przesyłek co noc eksportuje przesyłki z danego dnia jako dokumenty **JSON** (jeden dokument na przesyłkę). Każdy dokument jest zagnieżdżony: zawiera dane klienta z adresem, parametry paczki, obiekt z opcjami usługi, tablicę kuponów, tablicę pozycji (co jest w paczce) oraz tablicę zdarzeń śledzenia (CREATED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED).

Zespół danych ma przygotować dla działu operacyjnego i sprzedaży:

- spłaszczenie dokumentów JSON do postaci relacyjnej (pozycje, zdarzenia, opcje),
- bezpieczny proces ładowania kolejnego dnia z kontrolą jakości w tabelach tymczasowych – system źródłowy czasem wysyła ponownie przesyłki, które już mamy,
- szybkie, zawsze aktualne agregaty sprzedaży dla dashboardu (widoki zmaterializowane),
- deklaratywny pipeline KPI doręczeń, który odświeża się sam, bez pisania streamów i tasków (dynamiczne tabele).

Każdy moduł kończy się zapytaniem weryfikacyjnym, które zwraca listę testów ze statusem **PASS** lub **FAIL**.

1.2. Przykładowy dokument JSON

Tak wygląda jedna przesyłka po wygenerowaniu (dokument skrócony):

```
{
  "shipment_id": "PG-000010",
  "created_at": "2026-11-02T09:41:17",
  "service": "EXPRESS",
  "customer": { "id": "K00217", "segment": "B2C",
                "address": { "city": "Krakow", "zip": "31-552" } },
  "parcel":   { "weight_kg": 4.2, "dims_cm": [40, 30, 22] },
  "options":  { "insurance": true, "sms_notify": false, "cod_amount": 189.90 },
  "coupons":  [ "RABAT10" ],
  "items": [
    { "line": 1, "sku": "EL-101", "name": "Powerbank 20000 mAh",
      "category": "Elektronika", "qty": 2, "unit_price": 149 },
    { "line": 2, "sku": "DM-301", "name": "Kubek termiczny",
      "category": "Dom", "qty": 1, "unit_price": 59 }
  ],
  "events": [
    { "status": "CREATED",   "ts": "2026-11-02T09:41:17", "location": "Nadawca" },
    { "status": "PICKED_UP", "ts": "2026-11-02T17:55:02", "location": "Oddzial Krakow" },
    { "status": "IN_TRANSIT","ts": "2026-11-03T02:12:40", "location": "Sortownia Strykow" }
  ]
}
```

Struktura celowo zawiera elementy, które w praktyce sprawiają kłopot: zagnieżdżone obiekty, tablice obiektów, tablice puste (`coupons: []`), klucze występujące tylko czasem (`cod_amount`) i dwie niezależne tablice w jednym dokumencie (`items` i `events`).

1.3. Architektura rozwiązania

Moduł	Obiekt	Rola
Setup	`@RAW.LANDING_STAGE/shipments/`	Stage z plikami NDJSON (po jednym na dzień)
1. FLATTEN	`RAW.SHIPMENTS_RAW` (VARIANT)	Surowe dokumenty JSON załadowane przez COPY INTO
1. FLATTEN	`CORE.SHIPMENT_ITEMS`	Pozycje przesyłek spłaszczone przez LATERAL FLATTEN

Moduł	Obiekt	Rola
2. Tabele tymczasowe	`RAW.TMP_SHIPMENTS_DAY`, `RAW.TMP_NEW_SHIPMENTS`	Strefa robocza ładowania z kontrolą jakości (TEMPORARY)
2. Tabele tymczasowe	`UTIL.DQ_LOG`	Dziennik kontroli jakości (TRANSIENT, retencja 0)
3. Widoki zmaterializowane	`MART.MV_SHIPMENTS_FLAT`, `MART.MV_SALES_BY_CATEGORY`	Utrwalone projekcje JSON i agregaty sprzedaży
4. Dynamiczne tabele	`CORE.DT_SHIPMENTS`, `CORE.DT_SHIPMENT_EVENTS`, `MART.DT_DELIVERY_KPI`	Deklaratywny pipeline KPI doręczeń z TARGET_LAG

Przepływ danych: **Pliki JSON → Stage → COPY INTO → SHIPMENTS_RAW → (FLATTEN) → CORE → MART**. Moduł 2 dokłada kontrolowane ładowanie przez tabele tymczasowe, moduł 3 przyspiesza odczyty, a moduł 4 zastępuje ręczny pipeline grafem dynamicznych tabel.

1.4. Agenda

Część	Weryfikacja
Wprowadzenie i przygotowanie środowiska (generowanie JSON)	Test startowy
Moduł 1. Dane półstrukturalne i FLATTEN	5 testów
Moduł 2. Tabele tymczasowe i transient – kontrolowane ładowanie	5 testów
Moduł 3. Widoki zmaterializowane	5 testów
Moduł 4. Dynamiczne tabele	5 testów
Podsumowanie i sprzątanie	–

1.5. Wymagania

- Konto Snowflake i dostęp do Snowsight (Worksheets). Pracujemy jako rola **SYSADMIN**.
- **Widoki zmaterializowane wymagają edycji Enterprise lub wyższej.** Konto trial zakładane domyślnie jest w edycji Enterprise. Na koncie Standard moduł 3 można pominąć – moduły 2 i 4 od niego nie zależą.
- Nie są potrzebne pliki zewnętrzne – dane JSON generuje skrypt startowy w Snowflake.
- **Cały warsztat wykonuj w jednym arkuszu (worksheet).** Tabele tymczasowe żyją tylko w sesji, a każdy arkusz w Snowsight to osobna sesja. Wyjątkiem jest jedno celowe ćwiczenie w module 2.

1.6. Oczekiwane liczby – ściąga dla trenera

Liczby pozycji, zdarzeń, kuponów i opcji wynikają z numeru przesyłki (stałe reguły generatora), więc są powtarzalne. Losowe są tylko wartości (miasta, produkty, ilości, wagi).

Moment	SHIPMENTS_RAW	SHIPMENT_ITEMS	Pozostałe
Po module 1 (plik 02.11)	300	600	zdarzeń 1050, kuponów 30, opcji 660
Po module 2 (plik 03.11: 305 dok., w tym 5 duplikatów)	600	1200	TMP_SHIPMENTS_DAY 305, TMP_NEW_SHIPMENTS 300
Po module 3 (korekta i usunięcie PG-000002)	600	1197	MV_SHIPMENTS_FLAT 600

Moment	SHIPMENTS_RAW	SHIPMENT_ITEMS	Pozostałe
Po module 4 (plik 04.11, 200 dok.)	800	1197	DT_SHIPMENTS 800, DT_SHIPMENT_EVENTS 2800, doręczonych 200

2. Przygotowanie środowiska (start od zera)

Skrypt tworzy warehouse, bazę i schematy, a następnie generuje 800 przesyłek z trzech dni jako dokumenty JSON (`OBJECT_CONSTRUCT`, `ARRAY_AGG`). Na koniec wyładowuje do stage dwa pliki NDJSON: z 02.11 (300 przesyłek) oraz z 03.11 (300 nowych i 5 przesyłek wysłanych ponownie). Plik z 04.11 „dotrze" w module 4.

Skopiuj cały skrypt do nowego arkusza i uruchom go w całości (**Run All**, Ctrl+Shift+Enter).

Krok 0.1. Warehouse, baza i schematy

```
USE ROLE SYSADMIN;

CREATE OR REPLACE WAREHOUSE PARCELGO_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE DATABASE PARCELGO;
CREATE SCHEMA PARCELGO.RAW;    -- stage i surowe dokumenty JSON
CREATE SCHEMA PARCELGO.CORE;   -- dane spłaszczone
CREATE SCHEMA PARCELGO.MART;   -- warstwa raportowa (MV, KPI)
CREATE SCHEMA PARCELGO.UTIL;   -- generator danych i dziennik kontroli jakości

USE WAREHOUSE PARCELGO_WH;
USE DATABASE PARCELGO;
```

Krok 0.2. Format pliku JSON i stage

```
CREATE OR REPLACE FILE FORMAT PARCELGO.RAW.FF_JSON
  TYPE = JSON
  STRIP_OUTER_ARRAY = TRUE   -- obsłuży też plik z jedną dużą tablicą [...]
  COMPRESSION = AUTO;

CREATE OR REPLACE STAGE PARCELGO.RAW.LANDING_STAGE
  FILE_FORMAT = PARCELGO.RAW.FF_JSON
  COMMENT = 'Nocne eksporty przesylek z systemu sledzenia (NDJSON)';
```

Krok 0.3. Katalog produktów (do generatora)

```
CREATE OR REPLACE TABLE PARCELGO.UTIL.PRODUCTS (
  product_no   INTEGER,
  sku          VARCHAR(20),
  product_name VARCHAR(100),
  category     VARCHAR(30),
  unit_price   NUMBER(10,2)
);

INSERT INTO PARCELGO.UTIL.PRODUCTS VALUES
  (1, 'BK-001', 'Ksiazka: Snowflake w praktyce', 'Ksiazki',      79.00),
  (2, 'EL-101', 'Powerbank 20000 mAh',           'Elektronika', 149.00),
  (3, 'EL-102', 'Sluchawki Bluetooth',           'Elektronika', 229.00),
  (4, 'OD-201', 'Kurtka przeciwdeszczowa',       'Odziez',      259.00),
  (5, 'OD-202', 'Buty trekkingowe',              'Odziez',      399.00),
  (6, 'DM-301', 'Kubek termiczny',               'Dom',          59.00),
  (7, 'DM-302', 'Lampka LED',                    'Dom',         119.00),
  (8, 'ZB-401', 'Klocki konstrukcyjne',          'Zabawki',     189.00);
```

Krok 0.4. Dane bazowe 800 przesyłek

Reguły generatora (stałe, aby testy były powtarzalne): liczba pozycji = `MOD(rn,3)+1`, liczba zdarzeń = `MOD(rn,4)+2` (tylko przesyłki z 5 zdarzeniami są doręczone), kupon ma co 10. przesyłka, pobranie (COD) co 5. przesyłka.

```
CREATE OR REPLACE TABLE PARCELGO.UTIL.SHIPMENT_BASE AS
WITH g AS (
  SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) AS rn
  FROM TABLE(GENERATOR(ROWCOUNT => 800))
),
d AS (
  SELECT rn,
         CASE WHEN rn <= 300 THEN '2026-11-02'::DATE
              WHEN rn <= 600 THEN '2026-11-03'::DATE
              ELSE '2026-11-04'::DATE END AS ship_date
  FROM g
)
SELECT
  rn,
  ship_date,
  'PG-' || LPAD(rn::VARCHAR, 6, '0')                                AS shipment_id,
  DATEADD(second, UNIFORM(21600, 64800, RANDOM()),
          ship_date::TIMESTAMP_NTZ)                                 AS created_ts,
  MOD(rn, 3) + 1                                                    AS item_cnt,
  MOD(rn, 4) + 2                                                    AS event_cnt,
  DECODE(UNIFORM(1, 3, RANDOM()), 1, 'STANDARD', 2, 'EXPRESS', 'ECONOMY') AS service,
  'K' || LPAD(UNIFORM(1, 500, RANDOM())::VARCHAR, 5, '0')           AS customer_id,
  IFF(UNIFORM(1, 4, RANDOM()) = 1, 'B2B', 'B2C')                    AS segment,
  DECODE(UNIFORM(1, 6, RANDOM()), 1, 'Warszawa', 2, 'Krakow', 3, 'Wroclaw',
                                  4, 'Poznan',   5, 'Gdansk', 'Lodz') AS city,
  INSERT(LPAD(UNIFORM(0, 99999, RANDOM())::VARCHAR, 5, '0'), 3, 0, '-') AS zip,
  (UNIFORM(1, 300, RANDOM()) / 10)::NUMBER(5,1)                     AS weight_kg,
  UNIFORM(10, 60, RANDOM())                                         AS dim_l,
  UNIFORM(10, 60, RANDOM())                                         AS dim_w,
  UNIFORM(5, 40, RANDOM())                                          AS dim_h,
  UNIFORM(0, 1, RANDOM()) = 1                                       AS insurance,
  UNIFORM(0, 1, RANDOM()) = 1                                       AS sms_notify,
  (UNIFORM(2000, 50000, RANDOM()) / 100)::NUMBER(8,2)               AS cod_amount
FROM d;
```

Krok 0.5. Tablice pozycji i zdarzeń

```
-- Pozycje: 1-3 na przesyłkę, zebrane do tablicy obiektów
CREATE OR REPLACE TABLE PARCELGO.UTIL.SHIPMENT_ITEMS_SRC AS
WITH idx AS (
  SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) AS line_no
  FROM TABLE(GENERATOR(ROWCOUNT => 3))
),
lines AS (
  SELECT b.rn, x.line_no,
         UNIFORM(1, 8, RANDOM()) AS product_no,
         UNIFORM(1, 4, RANDOM()) AS qty
  FROM PARCELGO.UTIL.SHIPMENT_BASE b
  JOIN idx x ON x.line_no <= b.item_cnt
)
SELECT l.rn,
       ARRAY_AGG(OBJECT_CONSTRUCT(
         'line', l.line_no, 'sku', p.sku, 'name', p.product_name,
         'category', p.category, 'qty', l.qty, 'unit_price', p.unit_price
       )) WITHIN GROUP (ORDER BY l.line_no) AS items
FROM lines l
JOIN PARCELGO.UTIL.PRODUCTS p ON p.product_no = l.product_no
GROUP BY l.rn;
```

```
-- Zdarzenia: 2-5 kolejnych statusów co ok. 8 godzin
CREATE OR REPLACE TABLE PARCELGO.UTIL.SHIPMENT_EVENTS_SRC AS
WITH idx AS (
  SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) - 1 AS step
  FROM TABLE(GENERATOR(ROWCOUNT => 5))
)
SELECT b.rn,
       ARRAY_AGG(OBJECT_CONSTRUCT(
         'status', GET(ARRAY_CONSTRUCT('CREATED', 'PICKED_UP', 'IN_TRANSIT',
                                       'OUT_FOR_DELIVERY', 'DELIVERED'),
                       x.step)::VARCHAR,
         'ts', TO_VARCHAR(
                 DATEADD(minute,
                         IFF(x.step = 0, 0,
                             x.step * 480 + UNIFORM(0, 120, RANDOM())),
                         b.created_ts),
                 'YYYY-MM-DD"T"HH24:MI:SS'),
         'location', DECODE(x.step, 0, 'Nadawca',
                                    1, 'Oddzial ' || b.city,
                                    2, 'Sortownia Strykow',
                                    3, 'Oddzial ' || b.city,
                                    'Odbiorca')
       )) WITHIN GROUP (ORDER BY x.step) AS events
FROM PARCELGO.UTIL.SHIPMENT_BASE b
JOIN idx x ON x.step < b.event_cnt
GROUP BY b.rn;
```

Krok 0.6. Złożenie dokumentów JSON

`OBJECT_CONSTRUCT` pomija klucze z wartością NULL – dzięki temu `cod_amount` pojawi się tylko w co piątej przesyłce, tak jak w prawdziwym systemie.

```
CREATE OR REPLACE TABLE PARCELGO.UTIL.SHIPMENTS_DOC AS
SELECT b.rn, b.ship_date,
       OBJECT_CONSTRUCT(
         'shipment_id', b.shipment_id,
         'created_at',  TO_VARCHAR(b.created_ts, 'YYYY-MM-DD"T"HH24:MI:SS'),
         'service',     b.service,
         'customer',    OBJECT_CONSTRUCT('id', b.customer_id, 'segment', b.segment,
                          'address', OBJECT_CONSTRUCT('city', b.city, 'zip', b.zip)),
         'parcel',      OBJECT_CONSTRUCT('weight_kg', b.weight_kg,
                          'dims_cm', ARRAY_CONSTRUCT(b.dim_l, b.dim_w, b.dim_h)),
         'options',     OBJECT_CONSTRUCT('insurance', b.insurance, 'sms_notify', b.sms_notify,
                          'cod_amount', IFF(MOD(b.rn, 5) = 0, b.cod_amount, NULL)),
         'coupons',     IFF(MOD(b.rn, 10) = 0, ARRAY_CONSTRUCT('RABAT10'), ARRAY_CONSTRUCT()),
         'items',       i.items,
         'events',      e.events
       ) AS doc
FROM PARCELGO.UTIL.SHIPMENT_BASE b
JOIN PARCELGO.UTIL.SHIPMENT_ITEMS_SRC  i ON i.rn = b.rn
JOIN PARCELGO.UTIL.SHIPMENT_EVENTS_SRC e ON e.rn = b.rn;
```

Krok 0.7. Wygenerowanie plików JSON w stage

```
-- Plik z 02.11.2026: 300 przesyłek
COPY INTO @PARCELGO.RAW.LANDING_STAGE/shipments/shipments_2026_11_02.json
FROM (SELECT doc FROM PARCELGO.UTIL.SHIPMENTS_DOC
      WHERE ship_date = '2026-11-02' ORDER BY rn)
FILE_FORMAT = (TYPE = JSON COMPRESSION = NONE)
SINGLE = TRUE OVERWRITE = TRUE;

-- Plik z 03.11.2026: 300 nowych + 5 przesyłek wysłanych ponownie (PG-000001..PG-000005)
COPY INTO @PARCELGO.RAW.LANDING_STAGE/shipments/shipments_2026_11_03.json
FROM (SELECT doc FROM PARCELGO.UTIL.SHIPMENTS_DOC
      WHERE ship_date = '2026-11-03' OR rn <= 5 ORDER BY rn)
FILE_FORMAT = (TYPE = JSON COMPRESSION = NONE)
SINGLE = TRUE OVERWRITE = TRUE;
```

Krok 0.8. Test startowy

```
SELECT 'Wygenerowano 800 dokumentow JSON' AS test,
       IFF((SELECT COUNT(*) FROM PARCELGO.UTIL.SHIPMENTS_DOC) = 800, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT 'Dokumenty zawieraja 1601 pozycji i 2800 zdarzen',
       IFF((SELECT SUM(ARRAY_SIZE(doc:items)) || '/' || SUM(ARRAY_SIZE(doc:events))
            FROM PARCELGO.UTIL.SHIPMENTS_DOC) = '1601/2800', 'PASS', 'FAIL')
UNION ALL
SELECT 'W stage sa 2 pliki z lacznie 605 dokumentami',
       IFF((SELECT COUNT(DISTINCT METADATA$FILENAME) || '/' || COUNT(*)
            FROM @PARCELGO.RAW.LANDING_STAGE/shipments/
                 (FILE_FORMAT => 'PARCELGO.RAW.FF_JSON')) = '2/605', 'PASS', 'FAIL');
```

Oczekiwany wynik: trzy wiersze ze statusem PASS.

3. Moduł 1. Dane półstrukturalne i FLATTEN

Cel modułu

Ładujesz JSON do kolumny VARIANT poleceniem COPY INTO z transformacją, odczytujesz pola notacją ścieżkową, rozumiesz kolumny wynikowe FLATTEN (SEQ, KEY, PATH, INDEX, VALUE, THIS), spłaszczasz tablice i obiekty, świadomie używasz OUTER => TRUE i unikasz mnożenia wierszy przy dwóch tablicach.

Krok 1.1. Podgląd plików w stage

```
USE ROLE SYSADMIN;
USE WAREHOUSE PARCELGO_WH;
USE SCHEMA PARCELGO.RAW;

LIST @LANDING_STAGE/shipments/;

-- Dla JSON cały dokument trafia do kolumny $1 (typ VARIANT)
SELECT METADATA$FILENAME AS plik, $1 AS dokument, $1:shipment_id::VARCHAR AS id
FROM @LANDING_STAGE/shipments/ (FILE_FORMAT => 'PARCELGO.RAW.FF_JSON')
LIMIT 3;
```

Krok 1.2. Tabela surowa z kolumną VARIANT

```
CREATE OR REPLACE TABLE PARCELGO.RAW.SHIPMENTS_RAW (
  doc       VARIANT,
  src_file  VARCHAR,
  loaded_at TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Krok 1.3. COPY INTO z transformacją – plik z 02.11

COPY może wykonać prosty SELECT na pliku: tu dokładamy nazwę pliku źródłowego (lineage). Kolumna `loaded_at` wypełni się wartością domyślną.

```
COPY INTO PARCELGO.RAW.SHIPMENTS_RAW (doc, src_file)
FROM (
  SELECT $1, METADATA$FILENAME
  FROM @PARCELGO.RAW.LANDING_STAGE/shipments/
)
FILES = ('shipments_2026_11_02.json')
FILE_FORMAT = (FORMAT_NAME = 'PARCELGO.RAW.FF_JSON');
```

Oczekiwany wynik: status `LOADED`, `rows_loaded` = 300.

Krok 1.4. Notacja ścieżkowa, rzutowanie i brakujące klucze

```
SELECT doc:shipment_id::VARCHAR              AS shipment_id,
       doc:created_at::TIMESTAMP_NTZ         AS created_ts,
       doc:customer.address.city::VARCHAR    AS city,          -- zagnieżdżony obiekt
       doc['customer']['segment']::VARCHAR   AS segment,       -- notacja nawiasowa
       doc:parcel.dims_cm[0]::INT            AS length_cm,     -- element tablicy
       ARRAY_SIZE(doc:items)                 AS item_cnt,
       doc:options.cod_amount                AS cod_raw,       -- brak klucza => NULL
       TYPEOF(doc:options.cod_amount)        AS cod_type,
       doc:service                           AS service_bez_rzutowania
FROM PARCELGO.RAW.SHIPMENTS_RAW
ORDER BY shipment_id
LIMIT 10;
```

Na co zwrócić uwagę: pole bez rzutowania wyświetla się w cudzysłowie (`"EXPRESS"`) – to wciąż VARIANT. Nazwy kluczy w ścieżce rozróżniają wielkość liter (`doc:Shipment_ID` zwróci NULL). Brakujący klucz nie powoduje błędu, tylko NULL – `TYPEOF` zwraca wtedy NULL, a dla istniejącej wartości np. `DECIMAL`.

Krok 1.5. Anatomia FLATTEN

FLATTEN to funkcja tabelaryczna, która z jednej tablicy lub obiektu robi wiele wierszy. Zobaczmy wszystkie kolumny, jakie zwraca:

```
SELECT f.seq, f.key, f.path, f.index, f.value, f.this
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:items) f
WHERE r.doc:shipment_id = 'PG-000002';
```

Kolumna	Znaczenie
`SEQ`	Numer rekordu wejściowego (unikalny dla wiersza źródłowego)
`KEY`	Nazwa klucza – wypełniona przy spłaszczaniu obiektu, NULL dla tablicy
`PATH`	Ścieżka do elementu, np. `[0]` lub `options.insurance`
`INDEX`	Pozycja w tablicy liczona od 0 – NULL dla obiektu
`VALUE`	Wartość elementu (VARIANT) – tu odwołujemy się dalej przez `:`
`THIS`	Cały spłaszczany element (tablica lub obiekt)

Krok 1.6. Spłaszczenie tablicy pozycji

```
SELECT r.doc:shipment_id::VARCHAR     AS shipment_id,
       i.index + 1                    AS line_no,
       i.value:sku::VARCHAR           AS sku,
       i.value:name::VARCHAR          AS product_name,
       i.value:category::VARCHAR      AS category,
       i.value:qty::INT               AS qty,
       i.value:unit_price::NUMBER(10,2) AS unit_price
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:items) i
ORDER BY shipment_id, line_no
LIMIT 20;

SELECT COUNT(*) AS pozycji
FROM PARCELGO.RAW.SHIPMENTS_RAW r, LATERAL FLATTEN(INPUT => r.doc:items) i;   -- 600
```

Krok 1.7. Spłaszczenie obiektu – opcje usługi jako pary klucz–wartość

FLATTEN działa także na obiekcie: każdy klucz staje się wierszem. To wygodny sposób na „odwrócenie” obiektu o zmiennym zestawie kluczy (np. atrybutów).

```
SELECT o.key AS opcja, TYPEOF(o.value) AS typ, COUNT(*) AS wystapien
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:options) o
GROUP BY 1, 2
ORDER BY 1;
```

Oczekiwany wynik: `cod_amount` (DECIMAL) – 60, `insurance` (BOOLEAN) – 300, `sms_notify` (BOOLEAN) – 300. Razem 660 wierszy.

Krok 1.8. Puste tablice i OUTER => TRUE

Tylko co dziesiąta przesyłka ma kupon. Domyślnie FLATTEN działa jak INNER JOIN – przesyłki z pustą tablicą znikają z wyniku. To częsty, cichy błąd w raportach.

```
-- Bez OUTER: zostają tylko przesyłki z kuponem
SELECT COUNT(*) AS wierszy
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:coupons) c;                    -- 30

-- Z OUTER: wszystkie przesyłki, kupon = NULL tam, gdzie go nie było
```

```
SELECT COUNT(*) AS wierszy, COUNT(c.value) AS z_kuponem
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:coupons, OUTER => TRUE) c;      -- 300 / 30
```

Krok 1.9. Pułapka: dwie tablice w jednym zapytaniu

```
SELECT COUNT(*) AS wierszy
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:items)  i,
     LATERAL FLATTEN(INPUT => r.doc:events) e;   -- 2100, a nie 600 ani 1050!
```

Dwa niezależne FLATTEN tworzą iloczyn kartezjański w obrębie przesyłki (pozycje × zdarzenia). Suma wartości pozycji policzona na takim wyniku byłaby kilkukrotnie zawyżona. Zasada: **każdą niezależną tablicę spłaszczamy w osobnym zapytaniu lub osobnej tabeli** i łączymy dopiero zagregowane wyniki.

Krok 1.10. FLATTEN rekurencyjny – mapa struktury dokumentu

```
SELECT f.path, TYPEOF(f.value) AS typ
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc, RECURSIVE => TRUE) f
WHERE r.doc:shipment_id = 'PG-000010'
ORDER BY f.path;
```

Tryb `RECURSIVE => TRUE` schodzi na wszystkie poziomy zagnieżdżenia. Przydaje się przy poznawaniu nieudokumentowanego JSON-a: jedno zapytanie pokazuje wszystkie ścieżki i typy.

Krok 1.11. Utrwalenie pozycji w warstwie CORE

```
CREATE OR REPLACE TABLE PARCELGO.CORE.SHIPMENT_ITEMS AS
SELECT r.doc:shipment_id::VARCHAR                    AS shipment_id,
       r.doc:created_at::TIMESTAMP_NTZ::DATE         AS ship_date,
       i.index + 1                                   AS line_no,
       i.value:sku::VARCHAR                          AS sku,
       i.value:name::VARCHAR                         AS product_name,
       i.value:category::VARCHAR                     AS category,
       i.value:qty::INT                              AS qty,
       i.value:unit_price::NUMBER(10,2)              AS unit_price,
       (i.value:qty::INT * i.value:unit_price::NUMBER(10,2))::NUMBER(12,2) AS line_value
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:items) i;
```

> **Wskazówka**
>
> Snowflake przy ładowaniu JSON automatycznie wydziela często występujące ścieżki do wewnętrznych kolumn (subcolumnarization). Dzięki temu zapytania po `doc:service` są szybkie i korzystają z przycinania mikropartycji. Nie dotyczy to jednak elementów tablic – stąd sens utrwalania spłaszczonych pozycji.
>
> Dla porównania z PostgreSQL: FLATTEN odpowiada `jsonb_array_elements` / `jsonb_each`, a `OUTER => TRUE` to odpowiednik `LEFT JOIN LATERAL … ON TRUE`.

Weryfikacja modułu 1

```
SELECT '1.1 SHIPMENTS_RAW zawiera 300 dokumentow' AS test,
       IFF((SELECT COUNT(*) FROM PARCELGO.RAW.SHIPMENTS_RAW) = 300, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '1.2 Wszystkie dokumenty maja plik zrodlowy i czas ladowania',
       IFF((SELECT COUNT(*) FROM PARCELGO.RAW.SHIPMENTS_RAW
            WHERE src_file LIKE '%shipments_2026_11_02.json'
              AND loaded_at IS NOT NULL) = 300, 'PASS', 'FAIL')
UNION ALL
SELECT '1.3 SHIPMENT_ITEMS = 600 = suma rozmiarow tablic items',
       IFF((SELECT COUNT(*) FROM PARCELGO.CORE.SHIPMENT_ITEMS) = 600
           AND (SELECT SUM(ARRAY_SIZE(doc:items)) FROM PARCELGO.RAW.SHIPMENTS_RAW) = 600,
           'PASS', 'FAIL')
```

```
UNION ALL
SELECT '1.4 Kupony: z OUTER 300 wierszy, bez OUTER 30',
       IFF((SELECT COUNT(*) FROM PARCELGO.RAW.SHIPMENTS_RAW r,
                   LATERAL FLATTEN(INPUT => r.doc:coupons, OUTER => TRUE) c) || '/' ||
           (SELECT COUNT(*) FROM PARCELGO.RAW.SHIPMENTS_RAW r,
                   LATERAL FLATTEN(INPUT => r.doc:coupons) c) = '300/30', 'PASS', 'FAIL')
UNION ALL
SELECT '1.5 Obiekt options daje 660 par klucz-wartosc',
       IFF((SELECT COUNT(*) FROM PARCELGO.RAW.SHIPMENTS_RAW r,
                   LATERAL FLATTEN(INPUT => r.doc:options) o) = 660, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli 1.1 pokazuje 600, załadowano oba pliki – uruchom ponownie krok 1.2 (CREATE OR REPLACE) i krok 1.3 z parametrem `FILES`.

4. Moduł 2. Tabele tymczasowe i transient – kontrolowane ładowanie

Cel modułu

Znasz różnice między tabelami permanent, transient i temporary (widoczność, czas życia, Time Travel, Fail-safe, koszt), ładujesz kolejny dzień przez tymczasową strefę roboczą z kontrolą jakości i odrzucasz duplikaty, zanim trafią do tabeli docelowej.

Trzy rodzaje tabel w Snowflake

Cecha	Permanent	Transient	Temporary
Widoczność	Wszyscy z uprawnieniami	Wszyscy z uprawnieniami	Tylko bieżąca sesja
Czas życia	Do DROP	Do DROP	Do końca sesji
Time Travel	0–1 dzień (Enterprise: do 90)	0–1 dzień	0–1 dzień
Fail-safe	7 dni	Brak	Brak
Typowe użycie	Dane produkcyjne	Staging, dane odtwarzalne, logi	Obliczenia pośrednie, strefa robocza ETL

Krok 2.1. Tymczasowa strefa robocza dla pliku z 03.11

```
USE SCHEMA PARCELGO.RAW;

CREATE OR REPLACE TEMPORARY TABLE PARCELGO.RAW.TMP_SHIPMENTS_DAY (
  doc      VARIANT,
  src_file VARCHAR
);

COPY INTO PARCELGO.RAW.TMP_SHIPMENTS_DAY (doc, src_file)
FROM (SELECT $1, METADATA$FILENAME FROM @PARCELGO.RAW.LANDING_STAGE/shipments/)
FILES = ('shipments_2026_11_03.json')
FILE_FORMAT = (FORMAT_NAME = 'PARCELGO.RAW.FF_JSON');

SELECT COUNT(*) FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY;   -- 305

SHOW TABLES LIKE 'TMP%' IN SCHEMA PARCELGO.RAW;          -- kolumna kind = TEMPORARY
```

Krok 2.2. Ćwiczenie: tabela tymczasowa jest widoczna tylko w sesji

Otwórz **nowy arkusz** (nowa sesja), ustaw kontekst i uruchom:

```
USE ROLE SYSADMIN; USE WAREHOUSE PARCELGO_WH;
SELECT COUNT(*) FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY;
-- Błąd: Object 'PARCELGO.RAW.TMP_SHIPMENTS_DAY' does not exist or not authorized.
```

Wróć do pierwszego arkusza – dalsza część warsztatu (łącznie z weryfikacją) musi działać w sesji, która utworzyła tabele tymczasowe.

Krok 2.3. Dziennik kontroli jakości jako tabela transient

Wyniki kontroli są odtwarzalne i nie potrzebują 7 dni Fail-safe, więc tworzymy tabelę **transient** z retencją 0 dni – nie płacimy za przechowywanie historii.

```
CREATE OR REPLACE TRANSIENT TABLE PARCELGO.UTIL.DQ_LOG (
  check_ts    TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP(),
  file_name   VARCHAR,
```

```
  check_name  VARCHAR,
  failed_rows NUMBER
)
DATA_RETENTION_TIME_IN_DAYS = 0;
```

Krok 2.4. Kontrola jakości na danych w strefie roboczej

```
INSERT INTO PARCELGO.UTIL.DQ_LOG (file_name, check_name, failed_rows)
SELECT 'shipments_2026_11_03.json', 'Przesylki juz obecne w RAW (duplikaty)',
       COUNT(*)
FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY t
WHERE EXISTS (SELECT 1 FROM PARCELGO.RAW.SHIPMENTS_RAW r
              WHERE r.doc:shipment_id::VARCHAR = t.doc:shipment_id::VARCHAR)
UNION ALL
SELECT 'shipments_2026_11_03.json', 'Duplikaty w obrebie pliku',
       COUNT(*) - COUNT(DISTINCT doc:shipment_id::VARCHAR)
FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY
UNION ALL
SELECT 'shipments_2026_11_03.json', 'Przesylki bez pozycji',
       COUNT_IF(ARRAY_SIZE(doc:items) = 0 OR doc:items IS NULL)
FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY
UNION ALL
SELECT 'shipments_2026_11_03.json', 'Brak shipment_id lub created_at',
       COUNT_IF(doc:shipment_id IS NULL OR doc:created_at IS NULL)
FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY;

SELECT check_name, failed_rows FROM PARCELGO.UTIL.DQ_LOG ORDER BY check_name;
```

Oczekiwany wynik: 5 duplikatów względem RAW (PG-000001…PG-000005), pozostałe kontrole = 0.

Krok 2.5. Druga tabela tymczasowa i zatwierdzenie ładowania w transakcji

```
-- Tylko przesyłki, których jeszcze nie mamy
CREATE OR REPLACE TEMPORARY TABLE PARCELGO.RAW.TMP_NEW_SHIPMENTS AS
SELECT t.*
FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY t
WHERE NOT EXISTS (SELECT 1 FROM PARCELGO.RAW.SHIPMENTS_RAW r
                  WHERE r.doc:shipment_id::VARCHAR = t.doc:shipment_id::VARCHAR);

SELECT COUNT(*) FROM PARCELGO.RAW.TMP_NEW_SHIPMENTS;   -- 300

BEGIN;
  INSERT INTO PARCELGO.RAW.SHIPMENTS_RAW (doc, src_file)
  SELECT doc, src_file FROM PARCELGO.RAW.TMP_NEW_SHIPMENTS;

  INSERT INTO PARCELGO.CORE.SHIPMENT_ITEMS
  SELECT n.doc:shipment_id::VARCHAR, n.doc:created_at::TIMESTAMP_NTZ::DATE,
         i.index + 1, i.value:sku::VARCHAR, i.value:name::VARCHAR,
         i.value:category::VARCHAR, i.value:qty::INT, i.value:unit_price::NUMBER(10,2),
         (i.value:qty::INT * i.value:unit_price::NUMBER(10,2))::NUMBER(12,2)
  FROM PARCELGO.RAW.TMP_NEW_SHIPMENTS n,
       LATERAL FLATTEN(INPUT => n.doc:items) i;
COMMIT;
```

Krok 2.6. Pułapka: tabela tymczasowa przesłania stałą o tej samej nazwie

```
CREATE OR REPLACE TABLE PARCELGO.UTIL.SETTINGS (param VARCHAR, val VARCHAR);
INSERT INTO PARCELGO.UTIL.SETTINGS VALUES ('env', 'PROD');

CREATE TEMPORARY TABLE PARCELGO.UTIL.SETTINGS (param VARCHAR, val VARCHAR);
INSERT INTO PARCELGO.UTIL.SETTINGS VALUES ('env', 'TEST');

SELECT * FROM PARCELGO.UTIL.SETTINGS;   -- TEST: sesja widzi tabelę tymczasową!

DROP TABLE PARCELGO.UTIL.SETTINGS;      -- usuwa tabelę tymczasową
SELECT * FROM PARCELGO.UTIL.SETTINGS;   -- PROD: stała tabela znowu widoczna
```

Snowflake pozwala utworzyć tabelę tymczasową o nazwie istniejącej tabeli stałej i w tej sesji ma ona pierwszeństwo. Dlatego tabele tymczasowe warto nazywać z prefiksem (np. `TMP_`).

Krok 2.7. Retencja 0 = brak Time Travel

```
SELECT * FROM PARCELGO.UTIL.DQ_LOG AT (OFFSET => -10);
-- Błąd: Time travel data is not available for table DQ_LOG ...

SHOW TABLES IN SCHEMA PARCELGO.UTIL;   -- kolumny kind i retention_time
```

> **Wskazówka**
>
> Tabeli transient nie da się zamienić na permanent poleceniem ALTER – trzeba utworzyć nową tabelę (CREATE TABLE … AS SELECT lub CLONE). Można natomiast tworzyć całe schematy i bazy TRANSIENT – wtedy wszystkie tabele w nich są transient.

Weryfikacja modułu 2

Uruchom w **tym samym arkuszu**, w którym tworzono tabele tymczasowe. Najpierw SHOW, a zaraz po nim zapytanie – `RESULT_SCAN(LAST_QUERY_ID())` czyta wynik poprzedniego polecenia.

```
SHOW TABLES IN DATABASE PARCELGO;

WITH t AS (
  SELECT "schema_name" AS s, "name" AS n, "kind" AS k, "retention_time" AS r
  FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
)
SELECT '2.1 SHIPMENTS_RAW: 600 dokumentow bez duplikatow' AS test,
       IFF((SELECT COUNT(*) || '/' || COUNT(DISTINCT doc:shipment_id::VARCHAR)
            FROM PARCELGO.RAW.SHIPMENTS_RAW) = '600/600', 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '2.2 SHIPMENT_ITEMS zawiera 1200 pozycji',
       IFF((SELECT COUNT(*) FROM PARCELGO.CORE.SHIPMENT_ITEMS) = 1200, 'PASS', 'FAIL')
UNION ALL
SELECT '2.3 TMP_SHIPMENTS_DAY i TMP_NEW_SHIPMENTS sa TEMPORARY (305 i 300 wierszy)',
       IFF((SELECT COUNT(*) FROM t WHERE n IN ('TMP_SHIPMENTS_DAY', 'TMP_NEW_SHIPMENTS')
                                    AND k = 'TEMPORARY') = 2
           AND (SELECT COUNT(*) FROM PARCELGO.RAW.TMP_SHIPMENTS_DAY) = 305
           AND (SELECT COUNT(*) FROM PARCELGO.RAW.TMP_NEW_SHIPMENTS) = 300, 'PASS', 'FAIL')
UNION ALL
SELECT '2.4 DQ_LOG jest TRANSIENT z retencja 0 dni',
       IFF((SELECT COUNT(*) FROM t WHERE s = 'UTIL' AND n = 'DQ_LOG'
                                    AND k = 'TRANSIENT' AND r::NUMBER = 0) = 1, 'PASS', 'FAIL')
UNION ALL
SELECT '2.5 DQ_LOG wykryl 5 duplikatow, pozostale kontrole = 0',
       IFF((SELECT SUM(failed_rows) || '/' ||
                   MAX(IFF(check_name ILIKE '%obecne w RAW%', failed_rows, NULL))
            FROM PARCELGO.UTIL.DQ_LOG) = '5/5', 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Test 2.3 daje FAIL po wylogowaniu lub odświeżeniu sesji – tabele tymczasowe zniknęły, co samo w sobie jest dobrą lekcją. Odtwórz je krokami 2.1 i 2.5 (bez bloku BEGIN…COMMIT). Jeśli 2.5 pokazuje np. `10/5`, kontrolę z kroku 2.4 uruchomiono dwa razy – wykonaj `TRUNCATE TABLE PARCELGO.UTIL.DQ_LOG` i powtórz krok 2.4.

5. Moduł 3. Widoki zmaterializowane

Cel modułu

Tworzysz widok zmaterializowany na danych JSON i widok agregujący, sprawdzasz, że MV jest zawsze aktualny mimo zmian w tabeli bazowej, znasz ograniczenia MV (jedna tabela, brak JOIN i FLATTEN) oraz wiesz, jak monitorować koszt jego utrzymania.

Wymagana edycja Enterprise

Na koncie w edycji Standard polecenie CREATE MATERIALIZED VIEW zakończy się błędem. Pomiń wtedy ten moduł – moduł 4 działa niezależnie od niego.

Krok 3.1. MV jako utrwalona projekcja JSON

Dashboard operacyjny wciąż odpytuje te same pola z dokumentów. Widok zmaterializowany przechowuje je jako zwykłe kolumny, a Snowflake w tle aktualizuje go po każdej zmianie w `SHIPMENTS_RAW`.

```
USE SCHEMA PARCELGO.MART;

CREATE OR REPLACE MATERIALIZED VIEW PARCELGO.MART.MV_SHIPMENTS_FLAT
  COMMENT = 'Najczesciej uzywane pola przesylki wyciagniete z JSON'
AS
SELECT doc:shipment_id::VARCHAR               AS shipment_id,
       TO_TIMESTAMP_NTZ(doc:created_at::VARCHAR, 'YYYY-MM-DD"T"HH24:MI:SS') AS created_ts,
       TO_DATE(LEFT(doc:created_at::VARCHAR, 10)) AS ship_date,
       doc:service::VARCHAR                   AS service,
       doc:customer.segment::VARCHAR          AS segment,
       doc:customer.address.city::VARCHAR     AS city,
       doc:parcel.weight_kg::NUMBER(5,1)      AS weight_kg,
       ARRAY_SIZE(doc:items)                  AS item_cnt,
       doc:options.insurance::BOOLEAN         AS insurance
FROM PARCELGO.RAW.SHIPMENTS_RAW;
```

Krok 3.2. MV agregujący sprzedaż według dnia i kategorii

```
CREATE OR REPLACE MATERIALIZED VIEW PARCELGO.MART.MV_SALES_BY_CATEGORY AS
SELECT ship_date,
       category,
       COUNT(*)        AS lines,
       SUM(qty)        AS units,
       SUM(line_value) AS revenue
FROM PARCELGO.CORE.SHIPMENT_ITEMS
GROUP BY ship_date, category;

SELECT * FROM PARCELGO.MART.MV_SALES_BY_CATEGORY ORDER BY ship_date, revenue DESC;

SHOW MATERIALIZED VIEWS IN SCHEMA PARCELGO.MART;
-- kolumny: rows, behind_by, refreshed_on, invalid, invalid_reason
```

Krok 3.3. MV jest zawsze aktualny – korekta i anulowanie

Dział obsługi zgłasza dwie zmiany: w przesyłce PG-000001 klient domówił po 10 sztuk każdej pozycji, a przesyłka PG-000002 została anulowana przed nadaniem.

```
SELECT SUM(lines), SUM(units) FROM PARCELGO.MART.MV_SALES_BY_CATEGORY;   -- przed zmianą

UPDATE PARCELGO.CORE.SHIPMENT_ITEMS SET qty = qty + 10,
       line_value = (qty + 10) * unit_price
WHERE shipment_id = 'PG-000001';
```

```
DELETE FROM PARCELGO.CORE.SHIPMENT_ITEMS WHERE shipment_id = 'PG-000002';

-- Natychmiast po zmianie - bez żadnego REFRESH
SELECT SUM(lines), SUM(units) FROM PARCELGO.MART.MV_SALES_BY_CATEGORY;   -- lines = 1197

SHOW MATERIALIZED VIEWS IN SCHEMA PARCELGO.MART;   -- behind_by może chwilę pokazywać opóźnienie
```

Dlaczego wynik jest aktualny, choć odświeżanie trwa w tle? Jeśli MV nie nadąża (`behind_by` > 0), Snowflake w czasie zapytania łączy zmaterializowane dane ze zmianami z tabeli bazowej. Użytkownik nigdy nie widzi nieaktualnego wyniku – inaczej niż w PostgreSQL, gdzie `REFRESH MATERIALIZED VIEW` trzeba wywołać samodzielnie.

Krok 3.4. Optymalizator może użyć MV sam

```
-- Zapytanie do TABELI, nie do widoku:
SELECT category, SUM(qty)
FROM PARCELGO.CORE.SHIPMENT_ITEMS
WHERE ship_date = '2026-11-03'
GROUP BY category;
```

Otwórz **Query Profile** tego zapytania. Optymalizator może automatycznie przepisać zapytanie tak, by czytało `MV_SALES_BY_CATEGORY` zamiast tabeli. Przy tak małej tabeli nie zawsze to zrobi – efekt jest wyraźny przy dużych danych.

Krok 3.5. Ograniczenia – czego MV nie potrafi

```
-- Próba 1: JOIN - błąd
CREATE OR REPLACE MATERIALIZED VIEW PARCELGO.MART.MV_BAD_JOIN AS
SELECT i.category, s.city, SUM(i.line_value) AS revenue
FROM PARCELGO.CORE.SHIPMENT_ITEMS i
JOIN PARCELGO.MART.MV_SHIPMENTS_FLAT s ON s.shipment_id = i.shipment_id
GROUP BY 1, 2;

-- Próba 2: FLATTEN - błąd
CREATE OR REPLACE MATERIALIZED VIEW PARCELGO.MART.MV_BAD_FLATTEN AS
SELECT r.doc:shipment_id::VARCHAR AS shipment_id, e.value:status::VARCHAR AS status
FROM PARCELGO.RAW.SHIPMENTS_RAW r, LATERAL FLATTEN(INPUT => r.doc:events) e;
```

Oczekiwany wynik: oba polecenia kończą się błędem `Invalid materialized view definition` (treść komunikatu wskazuje przyczynę). Właśnie dlatego w module 1 utrwaliliśmy pozycje w tabeli `SHIPMENT_ITEMS`, a w module 4 te ograniczenia znikną.

Ograniczenie MV	Konsekwencja
Tylko jedna tabela źródłowa	Brak JOIN, brak UNION, brak MV na innym widoku
Brak funkcji tabelarycznych	Nie można użyć FLATTEN ani UDTF
Ograniczone agregaty	SUM, COUNT, MIN, MAX, AVG są dozwolone; brak funkcji okienkowych, HAVING, ORDER BY
Tylko funkcje deterministyczne	Nie wolno użyć np. CURRENT_TIMESTAMP()
Koszt utrzymania	Odświeżanie w tle zużywa kredyty (serverless) i storage

Krok 3.6. Koszt utrzymania MV

```
SELECT *
FROM TABLE(PARCELGO.INFORMATION_SCHEMA.MATERIALIZED_VIEW_REFRESH_HISTORY(
       DATE_RANGE_START => DATEADD('hour', -2, CURRENT_TIMESTAMP())));
```

Dane w tej funkcji mogą pojawić się z kilkuminutowym opóźnieniem. Zbiorczo koszty widać w `SNOWFLAKE.ACCOUNT_USAGE.MATERIALIZED_VIEW_REFRESH_HISTORY`.

Wskazówka

Kiedy MV ma sens: zapytania wielokrotnie powtarzane, wynik dużo mniejszy niż tabela, tabela zmienia się rzadziej niż jest czytana. Przy często zmienianej tabeli koszt odświeżania może przewyższyć zysk.

MV może mieć własny klucz klastrowania (`ALTER MATERIALIZED VIEW … CLUSTER BY`), inny niż tabela bazowa – to sposób na drugi „porządek fizyczny" tych samych danych.

Weryfikacja modułu 3

```
SHOW MATERIALIZED VIEWS IN DATABASE PARCELGO;

WITH m AS (
  SELECT "name" AS n, "invalid"::VARCHAR AS inv
  FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
)
SELECT '3.1 Istnieja 2 poprawne widoki zmaterializowane' AS test,
       IFF((SELECT COUNT(*) FROM m WHERE n IN ('MV_SHIPMENTS_FLAT', 'MV_SALES_BY_CATEGORY')
                                    AND inv ILIKE 'false') = 2, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '3.2 MV_SHIPMENTS_FLAT ma tyle wierszy co SHIPMENTS_RAW (600)',
       IFF((SELECT COUNT(*) FROM PARCELGO.MART.MV_SHIPMENTS_FLAT) = 600
           AND (SELECT COUNT(*) FROM PARCELGO.RAW.SHIPMENTS_RAW) = 600, 'PASS', 'FAIL')
UNION ALL
SELECT '3.3 MV_SALES_BY_CATEGORY identyczny z agregacja ad hoc',
       IFF((SELECT COUNT(*) FROM (
              (SELECT ship_date, category, lines, units, revenue
               FROM PARCELGO.MART.MV_SALES_BY_CATEGORY
               EXCEPT
               SELECT ship_date, category, COUNT(*), SUM(qty), SUM(line_value)
               FROM PARCELGO.CORE.SHIPMENT_ITEMS GROUP BY 1, 2)
              UNION ALL
              (SELECT ship_date, category, COUNT(*), SUM(qty), SUM(line_value)
               FROM PARCELGO.CORE.SHIPMENT_ITEMS GROUP BY 1, 2
               EXCEPT
               SELECT ship_date, category, lines, units, revenue
               FROM PARCELGO.MART.MV_SALES_BY_CATEGORY))) = 0, 'PASS', 'FAIL')
UNION ALL
SELECT '3.4 MV uwzglednia usuniecie PG-000002: 1197 pozycji',
       IFF((SELECT SUM(lines) FROM PARCELGO.MART.MV_SALES_BY_CATEGORY) = 1197, 'PASS', 'FAIL')
UNION ALL
SELECT '3.5 MV_SHIPMENTS_FLAT: 2 dni i 1200 pozycji wg dokumentow',
       IFF((SELECT COUNT(DISTINCT ship_date) || '/' || SUM(item_cnt)
            FROM PARCELGO.MART.MV_SHIPMENTS_FLAT) = '2/1200', 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Test 3.5 celowo pokazuje 1200, a nie 1197: MV na dokumentach JSON nie wie o korekcie w tabeli `SHIPMENT_ITEMS` – to dwa różne źródła prawdy. Dobry punkt do dyskusji o tym, gdzie powinny trafiać korekty.

6. Moduł 4. Dynamiczne tabele

Cel modułu

Budujesz graf trzech dynamicznych tabel (z FLATTEN i JOIN – czego nie potrafi MV), rozumiesz TARGET_LAG i DOWNSTREAM, tryby odświeżania INCREMENTAL/FULL, odświeżasz tabele ręcznie, monitorujesz historię odświeżeń i porównujesz dynamiczne tabele ze streamem i taskiem.

Krok 4.1. Warstwa CORE: przesyłki i zdarzenia

Dynamiczna tabela to zapytanie SELECT plus deklaracja świeżości. Snowflake sam wykrywa zmiany w źródłach i odświeża wynik – w miarę możliwości przyrostowo. `TARGET_LAG = DOWNSTREAM` oznacza: odświeżaj tylko wtedy, gdy potrzebuje tego tabela dalej w grafie.

```
USE SCHEMA PARCELGO.CORE;

CREATE OR REPLACE DYNAMIC TABLE PARCELGO.CORE.DT_SHIPMENTS
  TARGET_LAG = DOWNSTREAM
  WAREHOUSE  = PARCELGO_WH
AS
SELECT doc:shipment_id::VARCHAR                AS shipment_id,
       doc:created_at::TIMESTAMP_NTZ           AS created_ts,
       doc:created_at::TIMESTAMP_NTZ::DATE     AS ship_date,
       doc:service::VARCHAR                    AS service,
       doc:customer.address.city::VARCHAR      AS city,
       ARRAY_SIZE(doc:items)                   AS item_cnt,
       ARRAY_SIZE(doc:events)                  AS event_cnt,
       -- ostatni element tablicy zdarzeń = bieżący status
       GET(GET(doc:events, ARRAY_SIZE(doc:events) - 1), 'status')::VARCHAR AS current_status,
       src_file
FROM PARCELGO.RAW.SHIPMENTS_RAW;

CREATE OR REPLACE DYNAMIC TABLE PARCELGO.CORE.DT_SHIPMENT_EVENTS
  TARGET_LAG = DOWNSTREAM
  WAREHOUSE  = PARCELGO_WH
AS
SELECT r.doc:shipment_id::VARCHAR   AS shipment_id,
       e.index + 1                  AS event_seq,
       e.value:status::VARCHAR      AS status,
       e.value:ts::TIMESTAMP_NTZ    AS event_ts,
       e.value:location::VARCHAR    AS location
FROM PARCELGO.RAW.SHIPMENTS_RAW r,
     LATERAL FLATTEN(INPUT => r.doc:events) e;
```

Krok 4.2. Warstwa MART: KPI doręczeń (JOIN + agregacja)

```
CREATE OR REPLACE DYNAMIC TABLE PARCELGO.MART.DT_DELIVERY_KPI
  TARGET_LAG = '1 minute'
  WAREHOUSE  = PARCELGO_WH
AS
SELECT s.ship_date,
       s.city,
       COUNT(*)                                        AS shipments,
       COUNT_IF(s.current_status = 'DELIVERED')        AS delivered,
       COUNT_IF(s.current_status <> 'DELIVERED')       AS in_progress,
       ROUND(AVG(DATEDIFF('minute', s.created_ts, d.delivered_ts)) / 60, 1) AS avg_delivery_h
FROM PARCELGO.CORE.DT_SHIPMENTS s
LEFT JOIN (
  SELECT shipment_id, MAX(event_ts) AS delivered_ts
  FROM PARCELGO.CORE.DT_SHIPMENT_EVENTS
  WHERE status = 'DELIVERED'
  GROUP BY shipment_id
) d ON d.shipment_id = s.shipment_id
GROUP BY s.ship_date, s.city;
```

```
SELECT * FROM PARCELGO.MART.DT_DELIVERY_KPI ORDER BY ship_date, city;
```

Tabele powstają od razu wypełnione (pierwsze odświeżenie wykonuje się przy tworzeniu). `TARGET_LAG = '1 minute'` na końcu grafu „ciągnie" za sobą tabele z `DOWNSTREAM`.

Krok 4.3. Graf i tryb odświeżania

```
SHOW DYNAMIC TABLES IN DATABASE PARCELGO;
SELECT "name", "target_lag", "refresh_mode", "refresh_mode_reason", "scheduling_state", "rows"
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Kolumna `refresh_mode` pokazuje, czy Snowflake odświeża tabelę przyrostowo (`INCREMENTAL`), czy przelicza ją w całości (`FULL`). `refresh_mode_reason` wyjaśnia, dlaczego wybrano FULL. Graf zależności zobaczysz w Snowsight: **Data » Databases » PARCELGO » MART » Dynamic Tables » DT_DELIVERY_KPI » zakładka Graph**.

Krok 4.4. Dynamiczna tabela nie przyjmuje DML

```
DELETE FROM PARCELGO.CORE.DT_SHIPMENTS WHERE shipment_id = 'PG-000001';
-- Błąd: DML operations are not supported on dynamic tables (lub podobny komunikat)
```

Zawartość dynamicznej tabeli wynika wyłącznie z jej zapytania. Korekty wprowadzamy w źródle – co rozwiązuje problem „dwóch źródeł prawdy" z modułu 3.

Krok 4.5. Dotarł plik z 04.11 – ładujemy tylko RAW

```
-- System śledzenia eksportuje kolejny dzień (200 przesyłek)
COPY INTO @PARCELGO.RAW.LANDING_STAGE/shipments/shipments_2026_11_04.json
FROM (SELECT doc FROM PARCELGO.UTIL.SHIPMENTS_DOC
      WHERE ship_date = '2026-11-04' ORDER BY rn)
FILE_FORMAT = (TYPE = JSON COMPRESSION = NONE)
SINGLE = TRUE OVERWRITE = TRUE;

-- Ładujemy wskazany plik do RAW
COPY INTO PARCELGO.RAW.SHIPMENTS_RAW (doc, src_file)
FROM (SELECT $1, METADATA$FILENAME FROM @PARCELGO.RAW.LANDING_STAGE/shipments/)
FILES = ('shipments_2026_11_04.json')
FILE_FORMAT = (FORMAT_NAME = 'PARCELGO.RAW.FF_JSON');
```

Pytanie do grupy: dlaczego używamy `FILES`, a nie `PATTERN = '.*shipments_.*[.]json'`? (Odpowiedź: plik z 03.11 załadowaliśmy do tabeli tymczasowej, a nie przez COPY do `SHIPMENTS_RAW`. Metadane ładowania tej tabeli go nie znają, więc COPY z wzorcem załadowałby go ponownie i powstałyby duplikaty.)

Krok 4.6. Odświeżenie – automatyczne albo ręczne

Wystarczy odczekać 1–2 minuty, aż KPI odświeży się zgodnie z `TARGET_LAG`. Na warsztacie przyspieszamy to ręcznym odświeżeniem, wykonywanym w kolejności grafu:

```
ALTER DYNAMIC TABLE PARCELGO.CORE.DT_SHIPMENTS        REFRESH;
ALTER DYNAMIC TABLE PARCELGO.CORE.DT_SHIPMENT_EVENTS  REFRESH;
ALTER DYNAMIC TABLE PARCELGO.MART.DT_DELIVERY_KPI     REFRESH;

SELECT ship_date, SUM(shipments) AS przesylek, SUM(delivered) AS doreczonych
FROM PARCELGO.MART.DT_DELIVERY_KPI
GROUP BY ship_date ORDER BY ship_date;     -- 300 / 300 / 200

SELECT COUNT(*) FROM PARCELGO.MART.MV_SHIPMENTS_FLAT;   -- MV też już widzi 800
```

Krok 4.7. Historia odświeżeń

```
SELECT name, state, refresh_action, refresh_trigger,
       data_timestamp, refresh_start_time, refresh_end_time, state_message
FROM TABLE(PARCELGO.INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY(
       NAME_PREFIX => 'PARCELGO.'))
```

```
ORDER BY refresh_start_time DESC;
```

`refresh_trigger` pokazuje przyczynę odświeżenia: `CREATION`, `MANUAL` lub `SCHEDULED`. `refresh_action` mówi, co faktycznie zrobiono: `INCREMENTAL`, `FULL` lub `NO_DATA` (źródło się nie zmieniło, więc nic nie liczono).

Krok 4.8. Zawieszenie grafu

```
ALTER DYNAMIC TABLE PARCELGO.MART.DT_DELIVERY_KPI SUSPEND;
-- Tabele z TARGET_LAG = DOWNSTREAM przestają się odświeżać, bo nikt ich nie potrzebuje
```

> **Wskazówka**
>
> Porównanie z warsztatem „Pipeline zamówień”: tam ten sam efekt wymagał tabeli docelowej, streamu, taska z warunkiem WHEN i ręcznego zarządzania kolejnością RESUME. Dynamiczna tabela to jedno polecenie CREATE – za cenę mniejszej kontroli nad logiką (np. brak MERGE z własnymi regułami, brak wywołań procedur).
>
> TARGET_LAG to maksymalne dopuszczalne opóźnienie danych, a nie harmonogram. Im krótszy lag, tym częstsze odświeżenia i wyższy koszt warehouse.
>
> Dynamiczne tabele są dostępne we wszystkich edycjach Snowflake, także Standard.

Weryfikacja modułu 4

```
SELECT '4.1 DT_SHIPMENTS: 800 unikalnych przesylek' AS test,
       IFF((SELECT COUNT(*) || '/' || COUNT(DISTINCT shipment_id)
            FROM PARCELGO.CORE.DT_SHIPMENTS) = '800/800', 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '4.2 DT_SHIPMENT_EVENTS: 2800 zdarzen',
       IFF((SELECT COUNT(*) FROM PARCELGO.CORE.DT_SHIPMENT_EVENTS) = 2800, 'PASS', 'FAIL')
UNION ALL
SELECT '4.3 DT_DELIVERY_KPI: 800 przesylek, 200 doreczonych',
       IFF((SELECT SUM(shipments) || '/' || SUM(delivered)
            FROM PARCELGO.MART.DT_DELIVERY_KPI) = '800/200', 'PASS', 'FAIL')
UNION ALL
SELECT '4.4 KPI zgodne z obliczeniem ad hoc z dokumentow JSON',
       IFF((SELECT COUNT(*) FROM (
              (SELECT ship_date, city, shipments, delivered, in_progress
               FROM PARCELGO.MART.DT_DELIVERY_KPI
               EXCEPT
               SELECT doc:created_at::TIMESTAMP_NTZ::DATE, doc:customer.address.city::VARCHAR,
                      COUNT(*), COUNT_IF(ARRAY_SIZE(doc:events) = 5),
                      COUNT_IF(ARRAY_SIZE(doc:events) < 5)
               FROM PARCELGO.RAW.SHIPMENTS_RAW GROUP BY 1, 2)
              UNION ALL
              (SELECT doc:created_at::TIMESTAMP_NTZ::DATE, doc:customer.address.city::VARCHAR,
                      COUNT(*), COUNT_IF(ARRAY_SIZE(doc:events) = 5),
                      COUNT_IF(ARRAY_SIZE(doc:events) < 5)
               FROM PARCELGO.RAW.SHIPMENTS_RAW GROUP BY 1, 2
               EXCEPT
               SELECT ship_date, city, shipments, delivered, in_progress
               FROM PARCELGO.MART.DT_DELIVERY_KPI))) = 0, 'PASS', 'FAIL')
UNION ALL
SELECT '4.5 Wszystkie 3 dynamiczne tabele maja udane odswiezenie',
       IFF((SELECT COUNT(DISTINCT name)
            FROM TABLE(PARCELGO.INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY(
                   NAME_PREFIX => 'PARCELGO.'))
            WHERE state = 'SUCCEEDED'
              AND name IN ('DT_SHIPMENTS', 'DT_SHIPMENT_EVENTS', 'DT_DELIVERY_KPI')) = 3,
           'PASS', 'FAIL');

-- Kontrola stanu (oczekiwane: DT_DELIVERY_KPI = SUSPENDED)
SHOW DYNAMIC TABLES IN SCHEMA PARCELGO.MART;
SELECT "name", "scheduling_state" FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Oczekiwany wynik: 5 × PASS i `DT_DELIVERY_KPI` w stanie `SUSPENDED`. Jeśli testy 4.1–4.3 pokazują 600, tabele nie zostały jeszcze odświeżone – wykonaj krok 4.6. Wartość 1100 i więcej oznacza, że plik z 03.11 załadowano drugi raz (PATTERN zamiast FILES). Usuń wtedy duplikaty z RAW albo odtwórz tabelę przez Time Travel: `CREATE OR REPLACE TABLE … CLONE … BEFORE (STATEMENT => '<id COPY>')`.

7. Podsumowanie i sprzątanie

7.1. Co zbudowaliśmy

Funkcja	Czego się nauczyliśmy
VARIANT i ścieżki	Ładowanie JSON przez COPY z transformacją, notacja `:` i `[ ]`, rzutowanie, brakujące klucze, TYPEOF
FLATTEN	Kolumny SEQ/KEY/PATH/INDEX/VALUE/THIS, tablice i obiekty, OUTER => TRUE, iloczyn przy dwóch tablicach, RECURSIVE
Temporary	Sesyjna strefa robocza ETL, kontrola jakości przed zapisem, przesłanianie nazw
Transient	Dane odtwarzalne bez Fail-safe, retencja 0 i jej konsekwencje
Materialized View	Utrwalona projekcja JSON i agregaty, zawsze aktualny wynik, automatyczne przepisywanie zapytań, ograniczenia i koszt
Dynamic Table	Deklaratywny pipeline z FLATTEN i JOIN, TARGET_LAG i DOWNSTREAM, INCREMENTAL/FULL, REFRESH, historia, SUSPEND

7.2. Kiedy co wybrać

Kryterium	Widok	Widok zmaterializowany	Dynamiczna tabela	Stream + Task
Przechowuje wynik	Nie	Tak	Tak	Tak (tabela docelowa)
Świeżość	Zawsze bieżąca	Zawsze bieżąca	Zgodna z TARGET_LAG	Zgodna z harmonogramem
JOIN / FLATTEN	Tak	Nie	Tak	Tak
Logika	SELECT	SELECT z jednej tabeli	SELECT	Dowolny SQL, MERGE, procedury
Edycja	Wszystkie	Enterprise+	Wszystkie	Wszystkie
Typowe użycie	Abstrakcja, bezpieczeństwo	Przyspieszenie powtarzalnych zapytań	Transformacje ELT, warstwy CORE/MART	Złożone CDC, SCD, efekty uboczne

7.3. Pytania kontrolne

1. Dlaczego zapytanie z dwoma LATERAL FLATTEN (pozycje i zdarzenia) zwróciło 2100 wierszy i jak poprawnie policzyć wartość przesyłek ze statusem DELIVERED?
2. Co się stanie z tabelami TMP_*, gdy zamkniesz przeglądarkę? Czy trzeba je usuwać ręcznie?
3. Dlaczego MV_SHIPMENTS_FLAT pokazał 1200 pozycji, a MV_SALES_BY_CATEGORY 1197? Jak uniknąć takiej rozbieżności?
4. Kiedy dynamiczna tabela przejdzie w tryb FULL i co to oznacza dla kosztów przy tabeli źródłowej z miliardem wierszy?

7.4. Sprzątanie

```
USE ROLE SYSADMIN;
ALTER DYNAMIC TABLE IF EXISTS PARCELGO.MART.DT_DELIVERY_KPI SUSPEND;
DROP DATABASE IF EXISTS PARCELGO;     -- usuwa też MV i dynamiczne tabele
DROP WAREHOUSE IF EXISTS PARCELGO_WH;
```

```
-- Tabele tymczasowe znikną same po zakończeniu sesji
```